Pseudocódigo del generador de horarios

Colegio Madres Dominicas (MMDD), Concepción

Taller de Gestión de Operaciones — Grupo 5

11 de noviembre de 2026

Este documento describe en pseudocódigo el motor de asignación implementado en el paquete `notebooks/horarios/`. La clase `MotorHorarios` (`motor.py`, método `generar`) encadena el modelo de sesiones (`modelo.py`), la heurística constructiva (`heuristicas/construccion_voraz.py`) y una metaheurística de mejora: la búsqueda local iterada, ILS (`metaheuristicas/busqueda_local_iterada.py`), o la búsqueda tabú iterada, ITS (`metaheuristicas/busqueda_tabu_iterada.py`, por defecto). El método construye el horario semanal de los 24 cursos regulares y la Educación Física de los 3 niveles de párvulos en tres fases: (i) *prepara* el modelo: fija las franjas de electivos de 3° y 4° medio, que comparten las secciones A y B, y corta la malla de cada curso en sesiones dobles de 90 minutos y simples de 45; (ii) ubica cada sesión con una heurística *constructiva voraz*, de la más restringida a la menos restringida, en la posición de menor costo; y (iii) repara las violaciones restantes con una *metaheurística*: ILS o ITS.

ILS e ITS usan los mismos operadores (`metaheuristicas/vecindario.py`): el mismo vecindario, formado por los intercambios de sesiones dentro de un curso, y la misma perturbación. Las dos repiten el ciclo *búsqueda local* $\rightarrow$ *perturbación* $\rightarrow$ *aceptación* y solo se diferencian en su búsqueda local. La de ILS es un descenso de mayor pendiente que se detiene en el primer óptimo local. La de ITS es una búsqueda tabú, que acepta movimientos que empeoran el costo y usa una memoria tabú para no deshacerlos.

Todas las restricciones se evalúan con una única función de costo, separada en *claves*. Cada clave agrupa las sesiones que compiten por un mismo recurso: un docente en un bloque, una asignatura en un día, un gimnasio en un bloque o una sala en un bloque. Su costo se actualiza de forma incremental cada vez que una sesión entra o sale.

Notación

- $\mathcal{C}$: los 27 cursos: 24 regulares (1° básico a 4° medio, secciones A y B) y 3 de párvulos (Prekínder A, Kínder A y B), de los que solo se programa la Educación Física.
- $\mathcal{D}$: días de lunes a viernes. Un bloque es un par $s = (d, b)$ con día d y número $b \in \{1, \ldots, 10\}$; $B_{c,d}$: bloques de la jornada del curso c el día d.
- $\mathcal{P} = \{(1,2), (3,4), (5,6), (7,8), (9,10)\}$: pares pedagógicos. Los recreos caen después de los bloques 2, 4, 6 y 8, así que un par nunca cruza un recreo.
- M_c: malla curricular del curso c, formada por tuplas (a, h, P, e): asignatura, horas semanales, conjunto de docentes (más de uno si hay co-docencia) y espacio.
- Sesión $\sigma = (c_\sigma, a_\sigma, P_\sigma, e_\sigma, \lambda_\sigma)$: curso, asignatura, docentes, espacio y largo $\lambda_\sigma \in \{1, 2\}$ en bloques. Las sesiones de electivos son *fijas* (en los bloques de la grilla 2026, la opción por defecto); el resto son *movibles*. κ_σ: a quién atienden los docentes de σ (el curso, o el nivel completo en un electivo compartido por A y B); $\varphi(a)$: familia de la asignatura a (English skills cuenta como Inglés).
- $\mathrm{pos}(\sigma)$: tupla de bloques que ocupa la sesión σ. V_c: celdas movibles del curso c (su jornada sin los bloques fijos); $Y_c(s)$: sesión que ocupa la celda $s \in V_c$, o nulo; Π_c: pares pedagógicos con sus dos celdas en V_c.
- Claves de restricción y sesiones registradas en cada una, $\mathrm{occ}(k)$:
  - $\mathrm{D}(p, s)$: sesiones que usan al docente p en el bloque s;
  - $\mathrm{F}(c, d, f)$: sesiones movibles del curso c de la familia f el día d;

- G(s): sesiones de Educación Física en el bloque s;
- R(r, s): sesiones que usan la sala de capacidad limitada r en el bloque s.

- z: costo total, $f(x) = W \cdot H(x)$; $v = H(x)$: número de violaciones duras; $\mathcal{M}$: claves con alguna violación dura; $\mathcal{B}$: claves con costo blando (hoy no hay criterios blandos y $\mathcal{B}$ queda vacío).
- Recintos deportivos: Gimnasio B para párvulos y 1° a 4° básico ($n_B(s)$ sesiones), Gimnasio A para 5° básico a 4° medio ($n_A(s)$ sesiones). Cada gimnasio recibe un curso por bloque.
- Movimiento π: intercambio de celdas de un curso (Algoritmo 7); π^{-1}: el intercambio inverso; $A(\pi)$: sesiones que mueve π.
- z^*, pos^*: costo y posiciones de la mejor solución encontrada en toda la búsqueda; t: segundos desde el inicio de la búsqueda; q: pasos (movimientos aplicados).
- $[\,P\,]$: vale 1 si la proposición P es verdadera y 0 en caso contrario.
- Valores usados por el programa: semilla $\omega = 3$ (valor por defecto de `main`); $K = 150\,000$ pasos; T: segundos máximos de la búsqueda (sin límite al generar, 30 s al comparar métodos); peso duro $W = 1000$; capacidad de la Sala English skills $\mathrm{cap} = 1$; perturbación de $k = 15$ intercambios (ILS e ITS); hasta $L = 10$ movimientos laterales seguidos en el descenso de ILS; permanencia tabú $\tau \in \{10, \ldots, 20\}$ y $L_T = 150$ pasos sin mejora para cerrar cada búsqueda tabú de ITS.

Algoritmo 1 GenerarHorario($\mathcal{C}, \mathcal{D}, B, M, \omega, K, T, MH$)

Entrada: $\mathcal{C}$: cursos; $\mathcal{D}$: días hábiles; $B_{c,d}$: bloques de la jornada del curso c el día d; M_c: malla curricular del curso c; ω: semilla aleatoria; K: pasos máximos y T: segundos máximos de la búsqueda; $MH \in \{\mathsf{ILS}, \mathsf{ITS}\}$: metaheurística de mejora.

Salida: Horario H que asigna a cada curso y bloque una lección (a, P, e), cubre exactamente las horas de M_c y minimiza el costo z; es factible si $v = 0$.

1: inicializar el generador aleatorio con la semilla ω // lo comparten las tres fases
2: Preparar // Algoritmo 2
3: ConstrucciónVoraz // heurística constructiva, Algoritmo 3
4: **if** $MH = \mathsf{ILS}$ **then**
5: $\quad \mathrm{ILS}(K, T)$ // Algoritmo 5
6: **else**
7: $\quad \mathrm{ITS}(K, T)$ // Algoritmo 6
8: **end if**
9: $H \leftarrow$ horario construido a partir de $\mathrm{pos}(\sigma)$ para toda sesión σ
10: **return** H

Algoritmo 2 PREPARAR()

Entrada: Cursos $\mathcal{C}$, jornadas B, mallas M y franjas de electivos de 3° y 4° medio.
Salida: Sesiones fijas registradas, celdas V_c, pares Π_c y lista de sesiones movibles.

$\Sigma \leftarrow \emptyset$; $z \leftarrow 0$; $v \leftarrow 0$; $\mathrm{occ}(\cdot) \leftarrow \emptyset$; $\mathcal{M} \leftarrow \emptyset$; $\mathcal{B} \leftarrow \emptyset$; $L_c \leftarrow \emptyset$, $\forall\, c \in \mathcal{C}$
for all nivel $n \in \{3°\text{ medio}, 4°\text{ medio}\}$, franja $j \in \{1,2,3\}$ de n **do** // grilla oficial 2026
  $P \leftarrow$ docentes de los grupos de la franja j // A y B mezclados en grupos
  for all periodo doble (d, b) de la franja j **do** // 3 periodos de 2 bloques
    crear la sesión fija σ con $P_\sigma = P$, $\lambda_\sigma = 2$ y $\kappa_\sigma =$ electivos de n
    $\mathrm{pos}(\sigma) \leftarrow ((d, b), (d, b+1))$; $\Sigma \leftarrow \Sigma \cup \{\sigma\}$
    $L_{n\,\mathrm{A}} \leftarrow L_{n\,\mathrm{A}} \cup \{(d, b), (d, b+1)\}$; $L_{n\,\mathrm{B}} \leftarrow L_{n\,\mathrm{B}} \cup \{(d, b), (d, b+1)\}$
  end for
end for
for all $c \in \mathcal{C}$ **do**
  $V_c \leftarrow \{(d, b) : d \in \mathcal{D},\ b \in B_{c,d}\} \setminus L_c$; $Y_c(s) \leftarrow$ nulo, $\forall\, s \in V_c$
  $\Pi_c \leftarrow \{((d, b), (d, b')) : d \in \mathcal{D},\ (b, b') \in \mathcal{P},\ (d, b) \in V_c,\ (d, b') \in V_c\}$
end for
for all $c \in \mathcal{C}$, $(a, h, P, e) \in M_c$ **con** a que no es electivo **do**
  if c es de párvulos **then**
    agregar a Σ h sesiones movibles $(c, a, P, e, 1)$
  else
    agregar a Σ $\lfloor h/2 \rfloor$ sesiones $(c, a, P, e, 2)$ y $(h \bmod 2)$ sesiones $(c, a, P, e, 1)$
  end if
end for
for all sesión fija $\sigma \in \Sigma$ **do**
  REGISTRAR$(\sigma, \mathrm{pos}(\sigma), +1)$ // Algoritmo 9
end for

Algoritmo 3 ConstrucciónVoraz()

Entrada: Sesiones Σ ya creadas y sesiones fijas registradas.
Salida: Cada sesión movible ubicada: $\mathrm{pos}(\sigma)$ definido y Y_c completo (solución inicial).

$\eta(p) \leftarrow$ horas semanales de grilla del docente p // cada franja de electivos suma 6
$\Lambda \leftarrow$ sesiones movibles ordenadas por: // de la más a la menos restringida
    λ_σ decreciente, $|P_\sigma|$ decreciente, $\max_{p \in P_\sigma} \eta(p)$ decreciente, primero $e_\sigma =$ Gimnasio, y desempate aleatorio
for all $\sigma \in \Lambda$, en orden **do**
    $c \leftarrow c_\sigma$
    if $\lambda_\sigma = 2$ **then**
        $O \leftarrow \{(s, s') \in \Pi_c : Y_c(s) = Y_c(s') = \text{nulo}\}$ // pares libres
    else
        $O \leftarrow \{(s) : s \in V_c,\ Y_c(s) = \text{nulo}\}$ // celdas libres
    end if
    barajar O al azar; $S^* \leftarrow$ nulo; $\Delta^* \leftarrow \infty$
    for all $S \in O$, en ese orden **do**
        $\Delta \leftarrow \textsc{Registrar}(\sigma, S, +1)$; $\textsc{Registrar}(\sigma, S, -1)$ // costo de ubicar σ en S
        if $\Delta < \Delta^*$ **then**
            $(S^*, \Delta^*) \leftarrow (S, \Delta)$
            if $\Delta = 0$ **then**
                break // primera posición sin costo
            end if
        end if
    end for
    if $S^* =$ nulo **then**
        error: no hay espacio en la jornada de c
    end if
    $Y_c(s) \leftarrow \sigma,\ \forall\, s \in S^*$; $\mathrm{pos}(\sigma) \leftarrow S^*$; $\textsc{Registrar}(\sigma, S^*, +1)$
end for

Algoritmo 4 Operadores comunes de ILS e ITS

Entrada: Solución actual completa (pos, Y, occ, z, v, $\mathcal{M}$, $\mathcal{B}$); mejor solución (z^*, pos^*); contadores q y t.
Salida: Vecindario, mejor movimiento, perturbación, registro de la mejor solución y criterio de parada.

function VECINDARIO // $N(x)$: intercambios de las sesiones en conflicto
  $\mathcal{K} \leftarrow \mathcal{M}$ **si** $\mathcal{M} \neq \emptyset$, **si no** $\mathcal{B}$ // primero las violaciones duras
  $\Sigma_C \leftarrow \{\sigma \in \mathrm{occ}(k) : k \in \mathcal{K},\ \sigma \text{ movible}\}$
  return $\{(c_\sigma, \pi) : \sigma \in \Sigma_C,\ \pi \in \textsc{Vecinos}(\sigma)\}$, sin repetidos // Algoritmo 7
end function

function MEJORMOVIMIENTO(N, Adm) // evalúa todo N; Adm: filtro opcional
  $\mu \leftarrow$ nulo; $\Delta^* \leftarrow \infty$; $e \leftarrow 0$
  for all $(c, \pi) \in N$ **do**
    $\Delta \leftarrow \textsc{Mover}(c, \pi)$; $\textsc{Mover}(c, \pi^{-1})$ // evalúa y deshace
    if Adm definido $\wedge\ \neg\, Adm(c, \pi, \Delta)$ **then**
      continue
    end if
    if $\Delta < \Delta^*$ **then**
      $(\mu, \Delta^*) \leftarrow ((c, \pi), \Delta)$; $e \leftarrow 1$
    else if $\Delta = \Delta^*$ **then**
      $e \leftarrow e + 1$
      con probabilidad $1/e$: $\mu \leftarrow (c, \pi)$ // empates al azar, uniforme
    end if
  end for
  return (μ, Δ^*)
end function

procedure PERTURBAR(k) // k intercambios al azar
  for $i \leftarrow 1$ **to** k **do**
    $c' \leftarrow$ curso aleatorio; $\sigma' \leftarrow$ sesión aleatoria de c'
    $\pi' \leftarrow$ intercambio aleatorio de $\textsc{Vecinos}(\sigma')$; $\textsc{Mover}(c', \pi')$
  end for
end procedure

procedure ACTUALIZARMEJOR // después de cada movimiento
  if $z < z^*$ **then**
    $z^* \leftarrow z$; $\mathrm{pos}^* \leftarrow \mathrm{pos}$ // se registran también t y q
  end if
end procedure

function PARAR // criterio de parada común
  return $z = 0 \vee t \geq T \vee q \geq K$
end function

Algoritmo 5 $\mathrm{ILS}(K, T)$ — búsqueda local iterada

Entrada: K: pasos máximos; T: segundos máximos; solución inicial de la construcción voraz; k: intercambios de cada perturbación; L: movimientos laterales seguidos permitidos.
Salida: Mejor horario encontrado; factible si termina con $v = 0$.

procedure DESCENSO // búsqueda local: descenso de mayor pendiente
  $l \leftarrow 0$
  while $\neg$ PARAR **do**
    $N \leftarrow$ VECINDARIO; $(\mu, \Delta) \leftarrow$ MEJORMOVIMIENTO(N, nulo) // sin filtro
    if $\mu = \text{nulo} \vee \Delta > 0 \vee (\Delta = 0 \wedge l \geq L)$ **then**
      return // óptimo local
    end if
    $l \leftarrow l + 1$ **si** $\Delta = 0$, **si no** $l \leftarrow 0$ // cuenta los laterales seguidos
    MOVER(μ); $q \leftarrow q + 1$; ACTUALIZARMEJOR
  end while
end procedure

$z^* \leftarrow z$; $\text{pos}^* \leftarrow \text{pos}$; $q \leftarrow 0$; iniciar el cronómetro t
DESCENSO // $s^* \leftarrow$ BúsquedaLocal(s_0)
$\bar{z} \leftarrow z$; $\overline{\text{pos}} \leftarrow \text{pos}$ // solución actual
while $\neg$ PARAR **do**
  PERTURBAR(k); ACTUALIZARMEJOR // $s' \leftarrow$ Perturbación(s^*)
  DESCENSO // $s^{*\prime} \leftarrow$ BúsquedaLocal(s')
  if $z \leq \bar{z}$ **then** // aceptación: mejor o igual
    $\bar{z} \leftarrow z$; $\overline{\text{pos}} \leftarrow \text{pos}$
  else if $\neg$ PARAR **then**
    RESTAURAR$(\overline{\text{pos}})$ // se rechaza y se vuelve a la actual
  end if
end while
if $z > z^*$ **then**
  RESTAURAR(pos^*) // se entrega la mejor solución
end if

Algoritmo 6 $\mathrm{ITS}(K, T)$ — búsqueda tabú iterada

Entrada: K: pasos máximos; T: segundos máximos; solución inicial de la construcción voraz; k: intercambios de cada perturbación; $\tau \in \{10, \ldots, 20\}$: permanencia tabú; L_T: pasos sin mejora que cierran cada búsqueda tabú.
Salida: Mejor horario encontrado; factible si termina con $v = 0$.

procedure BúsquedaTabú // búsqueda local de ITS
  $\Theta(\cdot) \leftarrow -1$ // lista tabú: paso hasta el que una sesión no vuelve a una posición
  $z_T \leftarrow z$; $\mathrm{pos}_T \leftarrow \mathrm{pos}$; $u \leftarrow 0$ // mejor de esta búsqueda y pasos sin mejora
  while $u < L_T \wedge \neg$ Parar **do**
    $Adm(c, \pi, \Delta) \equiv z + \Delta < z^* \ \vee \ \Theta(\sigma', \pi(\mathrm{pos}(\sigma'))) \leq q, \ \forall\, \sigma' \in A(\pi)$
      // aspiración: un movimiento tabú se admite si mejora la mejor solución
    $N \leftarrow$ Vecindario; $(\mu, \Delta) \leftarrow$ MejorMovimiento(N, Adm)
    if $\mu =$ nulo **then**
      break // todos los movimientos son tabú
    end if
    for all $\sigma' \in A(\mu)$ **do**
      $\Theta(\sigma', \mathrm{pos}(\sigma')) \leftarrow q + \tau$, con τ entero aleatorio en $\{10, \ldots, 20\}$
    end for
    Mover(μ); $q \leftarrow q + 1$; ActualizarMejor // se aplica aunque empeore
    if $z < z_T$ **then**
      $z_T \leftarrow z$; $\mathrm{pos}_T \leftarrow \mathrm{pos}$; $u \leftarrow 0$
    else
      $u \leftarrow u + 1$
    end if
  end while
  if $z > z_T$ **then**
    Restaurar(pos_T) // entrega la mejor solución de esta búsqueda
  end if
end procedure

$z^* \leftarrow z$; $\mathrm{pos}^* \leftarrow \mathrm{pos}$; $q \leftarrow 0$; iniciar el cronómetro t
BúsquedaTabú // $s^* \leftarrow$ BúsquedaTabú(s_0)
$\bar{z} \leftarrow z$; $\overline{\mathrm{pos}} \leftarrow \mathrm{pos}$ // solución actual
while $\neg$ Parar **do**
  Perturbar(k); ActualizarMejor // $s' \leftarrow$ Perturbación(s^*)
  BúsquedaTabú // $s^{*\prime} \leftarrow$ BúsquedaTabú(s')
  if $z \leq \bar{z}$ **then** // aceptación: mejor o igual
    $\bar{z} \leftarrow z$; $\overline{\mathrm{pos}} \leftarrow \mathrm{pos}$
  else if $\neg$ Parar **then**
    Restaurar$(\overline{\mathrm{pos}})$ // se rechaza y se vuelve a la actual
  end if
end while
if $z > z^*$ **then**
  Restaurar(pos^*) // se entrega la mejor solución
end if

Algoritmo 7 $\textsc{Vecinos}(\sigma)$

Entrada: σ: sesión movible del curso $c = c_\sigma$.
Salida: Conjunto N de intercambios π: permutaciones de celdas de V_c que mueven σ.

$N \leftarrow \emptyset$; $s_1 \leftarrow$ primera celda de $\mathrm{pos}(\sigma)$
if existe $\pi_1 \in \Pi_c$ que contiene a s_1 **then** // intercambio de pares
  for all $\pi_2 \in \Pi_c \setminus \{\pi_1\}$ **do**
    $N \leftarrow N \cup \{\pi_1 \leftrightarrow \pi_2\}$ // 1.ª celda con 1.ª celda y 2.ª con 2.ª
  end for
end if
if $\lambda_\sigma = 1$ **then** // intercambio simple
  for all $s_2 \in V_c \setminus \{s_1\}$ **con** $Y_c(s_2) = \text{nulo} \vee \lambda_{Y_c(s_2)} = 1$ **do**
    $N \leftarrow N \cup \{s_1 \leftrightarrow s_2\}$
  end for
end if
return N // un doble siempre viaja en un par completo

Algoritmo 8 Claves de restricción y costo de cada clave

Entrada: Sesión σ con bloques candidatos S, o clave k con sus sesiones $\mathrm{occ}(k)$.
Salida: Claves que toca σ en S, y costo y violaciones duras de una clave.

function $\textsc{Claves}(\sigma, S)$ // S: bloques de un mismo día d
  $Q \leftarrow \{\mathsf{D}(p, s) : s \in S,\ p \in P_\sigma\}$
  if $e_\sigma =$ Gimnasio **then**
    $Q \leftarrow Q \cup \{\mathsf{G}(s) : s \in S\}$
  end if
  if e_σ es una sala de capacidad limitada r **then**
    $Q \leftarrow Q \cup \{\mathsf{R}(r, s) : s \in S\}$
  end if
  if σ es movible **then**
    $Q \leftarrow Q \cup \{\mathsf{F}(c_\sigma, d, \varphi(a_\sigma))\}$ // una sesión por familia y día
  end if
  return Q
end function

function $\textsc{CostoClave}(k)$ // retorna (costo, violaciones duras)
  $O \leftarrow \mathrm{occ}(k)$
  if $k = \mathsf{D}(p, s)$ **then**
    $n \leftarrow \max\{0, |\{\kappa_\sigma : \sigma \in O\}| - 1\}$ // un electivo de A y B cuenta una vez
  else if $k = \mathsf{F}(c, d, f)$ **then**
    $n \leftarrow \max\{0, |O| - 1\}$
  else if $k = \mathsf{R}(r, s)$ **then**
    $n \leftarrow \max\{0, |O| - \mathrm{cap}(r)\}$
  else // $k = \mathsf{G}(s)$
    $n \leftarrow \max\{0, n_B(s) - 1\} + \max\{0, n_A(s) - 1\}$ // un curso por gimnasio
  end if
  return $(Wn,\ n)$
end function

Algoritmo 9 Operaciones incrementales sobre el horario

Entrada: Estado actual: pos, Y, occ, z, v, $\mathcal{M}$ y $\mathcal{B}$.
Salida: Estado consistente tras cada ubicación, retiro o intercambio, y la variación de costo Δ.

function $\textsc{Registrar}(\sigma, S, \pm 1)$ // agrega o quita σ de las celdas S
  $\Delta \leftarrow 0$
  for all $k \in \textsc{Claves}(\sigma, S)$ **do**
    $(c_0, n_0) \leftarrow \textsc{CostoClave}(k)$
    $\mathrm{occ}(k) \leftarrow \mathrm{occ}(k) \cup \{\sigma\}$ **o** $\mathrm{occ}(k) \setminus \{\sigma\}$, según el signo
    $(c_1, n_1) \leftarrow \textsc{CostoClave}(k)$; $\Delta \leftarrow \Delta + c_1 - c_0$; $v \leftarrow v + n_1 - n_0$
    actualizar la pertenencia de k a $\mathcal{M}$ (si $n_1 > 0$) y a $\mathcal{B}$ (si tiene costo blando)
  end for
  $z \leftarrow z + \Delta$
  return Δ
end function

function $\textsc{Mover}(c, \pi)$ // π: permutación de celdas de V_c
  $A(\pi) \leftarrow \{Y_c(s) : s \in \mathrm{dom}(\pi),\ Y_c(s) \neq \text{nulo}\}$; $\Delta \leftarrow 0$
  for all $\sigma \in A(\pi)$ **do**
    $\Delta \leftarrow \Delta + \textsc{Registrar}(\sigma, \mathrm{pos}(\sigma), -1)$
  end for
  $Y_c(\pi(s)) \leftarrow Y_c(s)$ para toda $s \in \mathrm{dom}(\pi)$, en simultáneo
  for all $\sigma \in A(\pi)$ **do**
    $\mathrm{pos}(\sigma) \leftarrow \pi(\mathrm{pos}(\sigma))$, ordenado; $\Delta \leftarrow \Delta + \textsc{Registrar}(\sigma, \mathrm{pos}(\sigma), +1)$
  end for
  return Δ
end function

procedure $\textsc{Restaurar}(\mathrm{pos}')$
  $\mathrm{occ}(\cdot) \leftarrow \emptyset$; $z \leftarrow 0$; $v \leftarrow 0$; $\mathcal{M} \leftarrow \emptyset$; $\mathcal{B} \leftarrow \emptyset$; $\mathrm{pos} \leftarrow \mathrm{pos}'$
  reconstruir Y_c a partir de pos para todo $c \in \mathcal{C}$
  for all $\sigma \in \Sigma$ **do**
    $\textsc{Registrar}(\sigma, \mathrm{pos}(\sigma), +1)$
  end for
end procedure